

ShinyVillage

Memoria del Proyecto Final de Ciclo

Asignatura Proyecto Intermodular DAM	Alumna Cristina García Quintero	Fecha Marzo 2026	Versión Entrega 3
Enlace al repositorio del proyecto			

Español

ShinyVillage es un videojuego de rol en dos dimensiones desarrollado con Unity y C#. El jugador dirige a un personaje desde una perspectiva cenital, gestiona su inventario, interactúa con el mapa a través de un sistema de tiles y administra múltiples partidas guardadas en una base de datos local SQLite. El proyecto aplica una arquitectura en capas con patrones Singleton y Repository, orientada al aprendizaje del desarrollo de videojuegos en un contexto académico.

English

ShinyVillage is a 2D role-playing game built with Unity and C#. The player controls a character from a top-down perspective, managing an inventory, interacting with a tile-based map, and handling multiple save files stored in a local SQLite database. The project follows a layered architecture using Singleton and Repository patterns, developed in an academic context as a practical exercise in video game programming.

0. Contenido

1. Descripción del proyecto.....	3
1.1 Introducción	3
1.2 Contexto del proyecto.....	3
1.3 Objetivo del proyecto	3
1.4 Marco legal.....	3
2. Acuerdo del proyecto	5
2.1 Requisitos funcionales y no funcionales.....	5
2.2 Planificación de tareas y temporalización.....	6
2.3 Metodología a seguir	6
2.4 Temporalización	6
2.5 Presupuesto	7
2.6 Contrato y pliego de condiciones/licencia	7
2.7 Análisis de riesgos	7
3. Análisis y diseño.....	8
3.1 Modelado de datos	8
3.2 Análisis y diseño funcional	10
3.3 Análisis y diseño de la interfaz	18
3.4 Análisis y diseño de la arquitectura.....	18
7. Referencias	22

1. Descripción del proyecto

1.1 Introducción

ShinyVillage es un videojuego de rol en dos dimensiones (RPG 2D) creado con el motor Unity y programado en C#. La propuesta principal es poner al jugador en la dirección de una aldea, donde tiene que administrar los recursos, cultivar su granja y examinar el ambiente desde un punto de vista cenital típico del género.

El jugador maneja a un personaje que se mueve sin restricciones por el mapa, recoge objetos del entorno, los guarda en un inventario y realiza acciones con aquellos elementos del terreno que son interactivos, como las celdas agrícolas. El personaje es seguido de manera constante por la cámara. Como el progreso se almacena en una base de datos local (SQLite), el jugador tiene la posibilidad de crear múltiples partidas independientes, reanudarlas cuando quiera o eliminarlas desde el menú.

El ciclo de juego proyectado incorpora procedimientos relacionados con la agricultura (cultivar, regar y recoger productos en tiles), manejo del inventario (recoger, amontonar y soltar artículos) e interacción con el mapa a través de un sistema de tiles clasificados según su condición.

1.2 Contexto del proyecto

El proyecto se desarrolla en un contexto académico, sin un cliente externo, donde el propio equipo actúa como desarrollador y receptor técnico del producto. El entorno se basa en Unity con C#, utilizando SQLite integrado mediante NuGetForUnity para la persistencia de datos, mientras que el control de versiones se gestiona con Git mediante ramas de desarrollo.

La solución elegida estructura el juego en sistemas independientes conectados mediante patrones de diseño. Se utiliza el patrón Singleton en gestores como GameManager, DatabaseManager y SaveGameManager. La persistencia se gestiona con SQLite y el patrón Repository separa el acceso a la base de datos de la lógica del juego.

El juego contempla como único tipo de usuario el jugador. Su rol le permite crear y modificar partidas desde el menú, mover al personaje e interactuar con el mapa. Toda la gestión interna es transparente para el jugador y opera de forma automática en segundo plano.

1.3 Objetivo del proyecto

ShinyVillage tiene la finalidad de ofrecer al jugador una experiencia de ocio digital tranquilo, de estilo casual basado en una simple gestión de recursos y un entorno de fantasía. La aplicación no tiene vocación comercial en este momento porque está enmarcada en un contexto de aprendizaje y desarrollo en técnicas de programación de videojuegos.

1.4 Marco legal

Al tratarse de un videojuego de entretenimiento en fase de desarrollo académico, sin distribución comercial ni recogida de datos personales identificativos, el marco legal aplicable es reducido.

Protección de datos (RGPD / LOPDGGD)

La aplicación almacena únicamente datos de juego — nombre de personaje, progreso o posición — de forma local en el dispositivo del usuario, sin transmisión a servidores externos. En su estado actual no se tratan datos personales en el

sentido del RGPD (Reglamento UE 2016/679) ni de la LOPDGDD (LO 3/2018). Si en el futuro se incorporasen cuentas de usuario o cualquier dato identificativo, la aplicación quedaría sujeta a dichas normas.

Propiedad intelectual

El proyecto ShinyVillage se distribuye bajo una **licencia personalizada basada en Apache License 2.0**, a la que se añade una cláusula de restricción de uso comercial. Cualquier persona puede usar, estudiar, modificar y redistribuir el software, pero **no puede hacerlo con fines comerciales** sin autorización expresa. Esta licencia no está aprobada por la OSI al incluir dicha restricción, pero es plenamente válida desde el punto de vista legal.

Clasificación por edades

En caso de distribución pública, el sistema PEGI clasificaría previsiblemente el juego como PEGI 3, dado su contenido de fantasía sin elementos violentos ni adultos.

2. Acuerdo del proyecto

2.1 Requisitos funcionales y no funcionales

Requisitos funcionales

Capacidades que el programa debe poder ofrecer al usuario final, en este caso el jugador.

ID	Requisito
RF-01	El jugador puede crear una nueva partida introduciendo un nombre de personaje y un nombre de slot
RF-02	El jugador puede cargar una partida guardada previamente desde el menú principal
RF-03	El jugador puede borrar una partida existente, con confirmación previa
RF-04	El jugador puede sobrescribir una partida con el estado actual de la sesión en curso
RF-05	El personaje se desplaza por el mapa en las cuatro direcciones con animación asociada
RF-06	El jugador puede recoger objetos del mundo e incorporarlos al inventario
RF-07	El jugador puede consultar su inventario y eliminar objetos, que reaparecen en el mapa
RF-08	El jugador puede interactuar con tiles del mapa pulsando la tecla de acción
RF-09	El estado de la granja (tiles, cultivos, fases de crecimiento) se guarda y recupera por partida

Requisitos no funcionales

Este tipo de requisitos definen las condiciones y estándares de rendimiento y las restricciones del sistema.

ID	Requisito
RNF-01	Los datos de partida se almacenan localmente en el dispositivo del usuario mediante SQLite
RNF-02	El sistema de guardado persiste entre escenas sin pérdida de datos
RNF-03	La arquitectura debe permitir añadir nuevos sistemas (cultivos, NPCs) sin modificar el núcleo
RNF-04	El tiempo de carga de una partida guardada no debe ser perceptible para el usuario (Duración de menos de 5s)
RNF-05	El juego debe ejecutarse en PC con un rendimiento estable bajo Unity, sin crasheos ni problemas de rendimiento como quedarse "congeado"

2.2 Planificación de tareas y temporalización

El proyecto se organiza en varias fases bien diferenciadas que permiten avanzar de forma progresiva en su desarrollo.

En la Fase 1, centrada en los fundamentos del proyecto, se lleva a cabo la configuración inicial del entorno de Unity, así como la organización de la estructura de carpetas. Además, se implementa el sistema de movimiento del personaje junto con el seguimiento de cámara, y se desarrolla el uso de Tilemaps junto con las primeras interacciones básicas dentro del entorno del juego.

En la Fase 2, dedicada al sistema de inventario, se diseña la clase Inventory y toda la lógica relacionada con la gestión de slots, incluyendo la posibilidad de añadir, apilar y eliminar objetos. También se desarrolla la interfaz de usuario del inventario mediante componentes como Inventory_UI y Slot_UI, y se implementa la mecánica que permite soltar objetos al mundo.

La Fase 3 se enfoca en la persistencia de datos y la base de datos. En esta etapa se integra SQLite utilizando NuGetForUnity, se crea un DatabaseManager encargado de la conexión, la creación de tablas y las operaciones CRUD, y se desarrollan repositorios específicos como SaveSlotRepository y PlayerRepository. Finalmente, se implementa el SaveGameManager como punto de entrada único para gestionar el guardado de la partida.

En la Fase 4, actualmente en desarrollo, se trabaja en el menú principal y la gestión de partidas. Esto incluye la creación del MainMenuManager, que controla acciones como iniciar una nueva partida, cargar partidas existentes o eliminarlas. También se desarrolla el SaveSlotUI, un prefab que representa cada partida con botones dinámicos, y se integra todo el flujo completo desde el menú hasta el juego y el sistema de guardado.

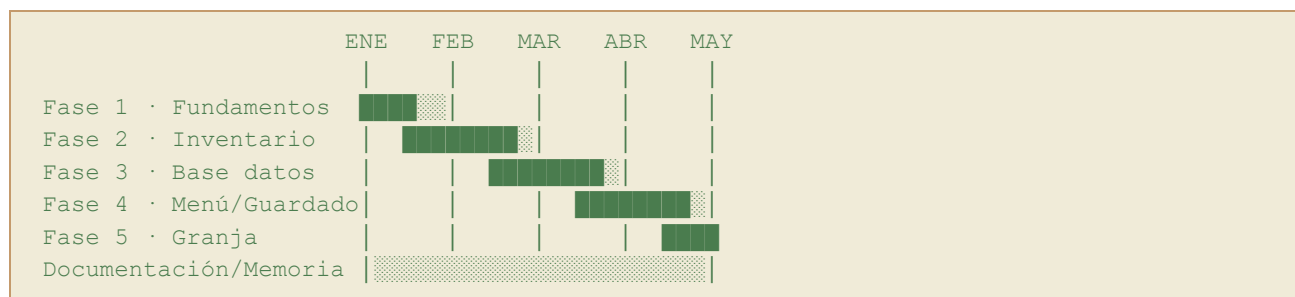
Por último, la Fase 5, también en desarrollo, aborda la implementación del sistema de granja. En ella se crean repositorios como FarmRepository y FarmTileRepository, se desarrolla la lógica de cultivo (plantar, regar y cosechar), y se garantiza la persistencia del estado de cada tile para cada partida.

2.3 Metodología a seguir

El proyecto adopta una metodología **iterativa e incremental** inspirada en Scrum, adaptada a un equipo de desarrollo pequeño y a un contexto académico. Cada fase equivale a un sprint con un entregable funcional al final. El desarrollo se gestiona mediante Git con ramas independientes por funcionalidad.

2.4 Temporalización

El siguiente diagrama refleja la distribución temporal estimada del proyecto:



Cada fase tiene una duración aproximada de dos a tres semanas, con solapamiento en la etapa de documentación, que se mantiene activa durante todo el desarrollo.

2.5 Presupuesto

En un contexto real de desarrollo profesional, este proyecto implicaría una serie de costes asociados tanto a los recursos humanos como a la infraestructura necesaria para su desarrollo.

El proyecto ha requerido una dedicación aproximada de 12 horas semanales desde el mes de enero hasta finales de curso (alrededor de 5 meses), lo que supone unas 240 horas de trabajo. Si se toma como referencia una tarifa media de un programador junior de aproximadamente 20 € por hora, el coste del desarrollo ascendería a una cantidad considerable.

Además, sería necesario contar con un equipo informático adecuado para trabajar con Unity de forma fluida, así como contemplar otros posibles costes relacionados con herramientas o recursos, aunque muchos de ellos pueden ser gratuitos.

A continuación, se muestra una estimación aproximada de los costes en un entorno real:

Concepto	Coste
Desarrollo de software (240h)	4.800 €
Equipo informático	1.000 €
SQLite + NuGetForUnity	0 €
Assets gráficos (libres de uso, CupNooble)	0 €
Control de versiones (Git y GitHub)	0 €
Unity (licencia personal)	0 €
Total	5.800 €

No obstante, en este caso concreto no ha existido ningún coste económico real, ya que el proyecto se ha desarrollado en un contexto académico, utilizando herramientas gratuitas o de uso libre, así como recursos previamente disponibles.

2.6 Contrato y pliego de condiciones/licencia

El proyecto se distribuye con una licencia Apache 2.0 con cláusula no comercial. Como se trata de un proyecto sin cliente externo, no hay contrato de prestación de servicios. El acuerdo de desarrollo queda recogido en los términos de entregas del centro formativo.

2.7 Análisis de riesgos

A lo largo del desarrollo del proyecto, se prevé la aparición de posibles situaciones que puedan afectar tanto a la planificación como a la correcta implementación del mismo. Por ello, se ha realizado un análisis de riesgos con el objetivo de identificar aquellos eventos potenciales, evaluar su probabilidad e impacto, y definir medidas de mitigación que permitan reducir sus efectos. A continuación, se presenta una tabla con los principales riesgos detectados durante el desarrollo del proyecto.

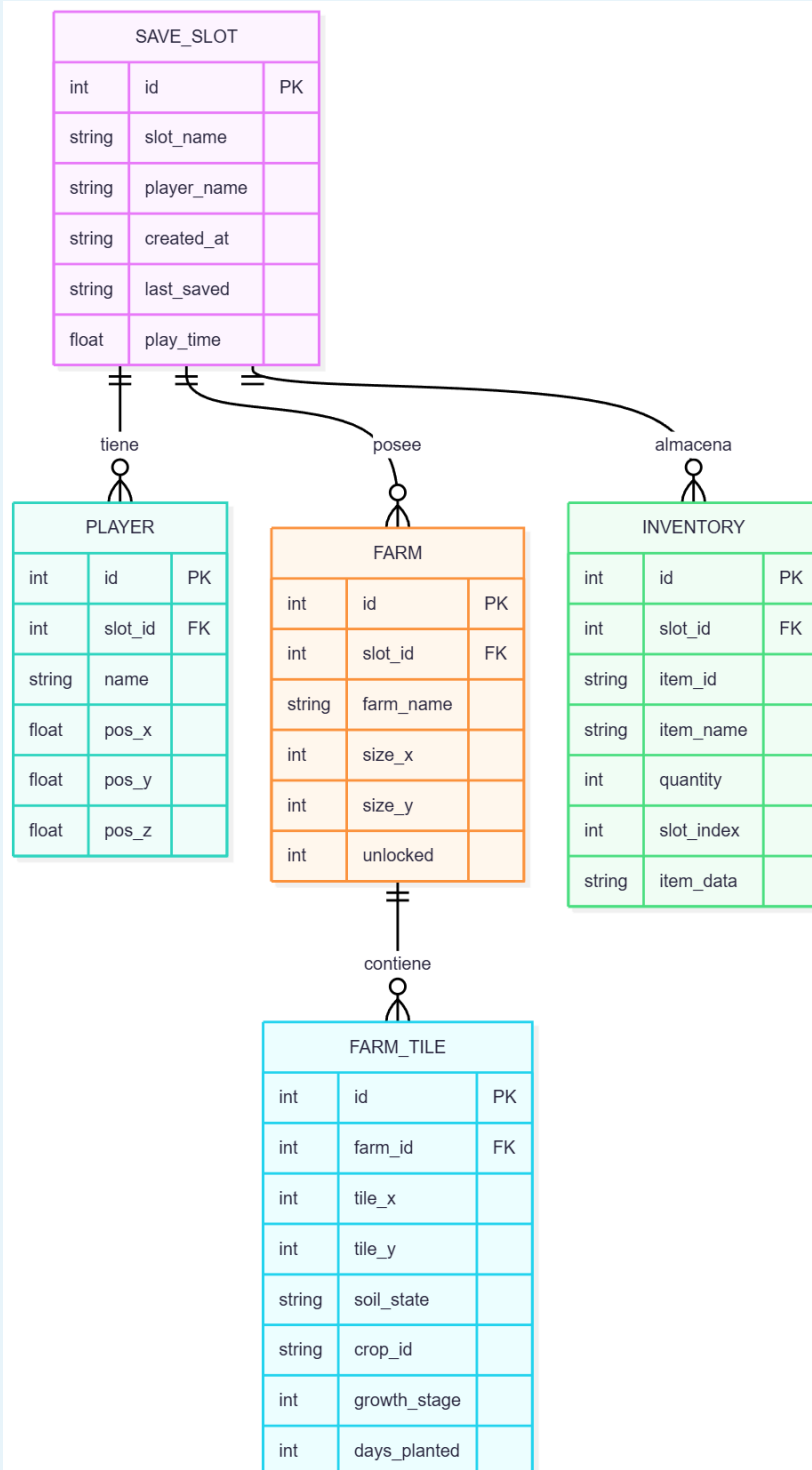
Riesgo	Probabilidad	Impacto	Mitigación
Corrupción de la base de datos SQLite	Baja	Alto	Cierre controlado de conexión en OnApplicationQuit y OnDestroy
Incompatibilidad de SQLite con la versión de Unity	Media	Alto	Uso de NuGetForUnity con versión fijada y probada
Deuda técnica por crecimiento no planificado	Media	Medio	Arquitectura en capas con patrón Repository desde el inicio
Pérdida de progreso en el repositorio Git	Baja	Alto	Ramas por funcionalidad y commits frecuentes
Falta de tiempo para completar el sistema de granja	Media	Medio	El núcleo es funcional sin él; se plantea como fase ampliable

3. Análisis y diseño

3.1 Modelado de datos

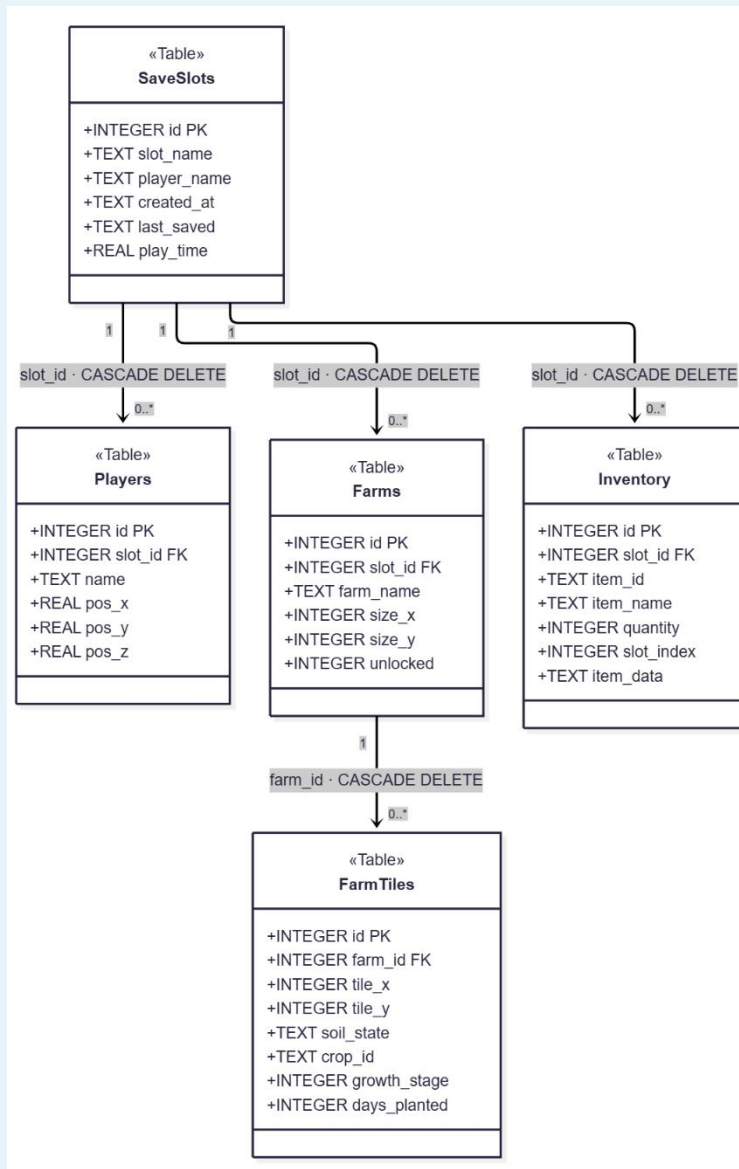
3.1.1 Modelo E/R

Diagrama:



3.1.2 Modelo relacional del sistema de guardado

Diagrama:



3.1.3 Script de la creación de la BBDD (SQLite): [Ver Anexos](#)

3.2 Análisis y diseño funcional

3.2.1 UML (Clases, secuencia y casos de uso)

Diagramas de clases

Diagrama general

Diagrama:

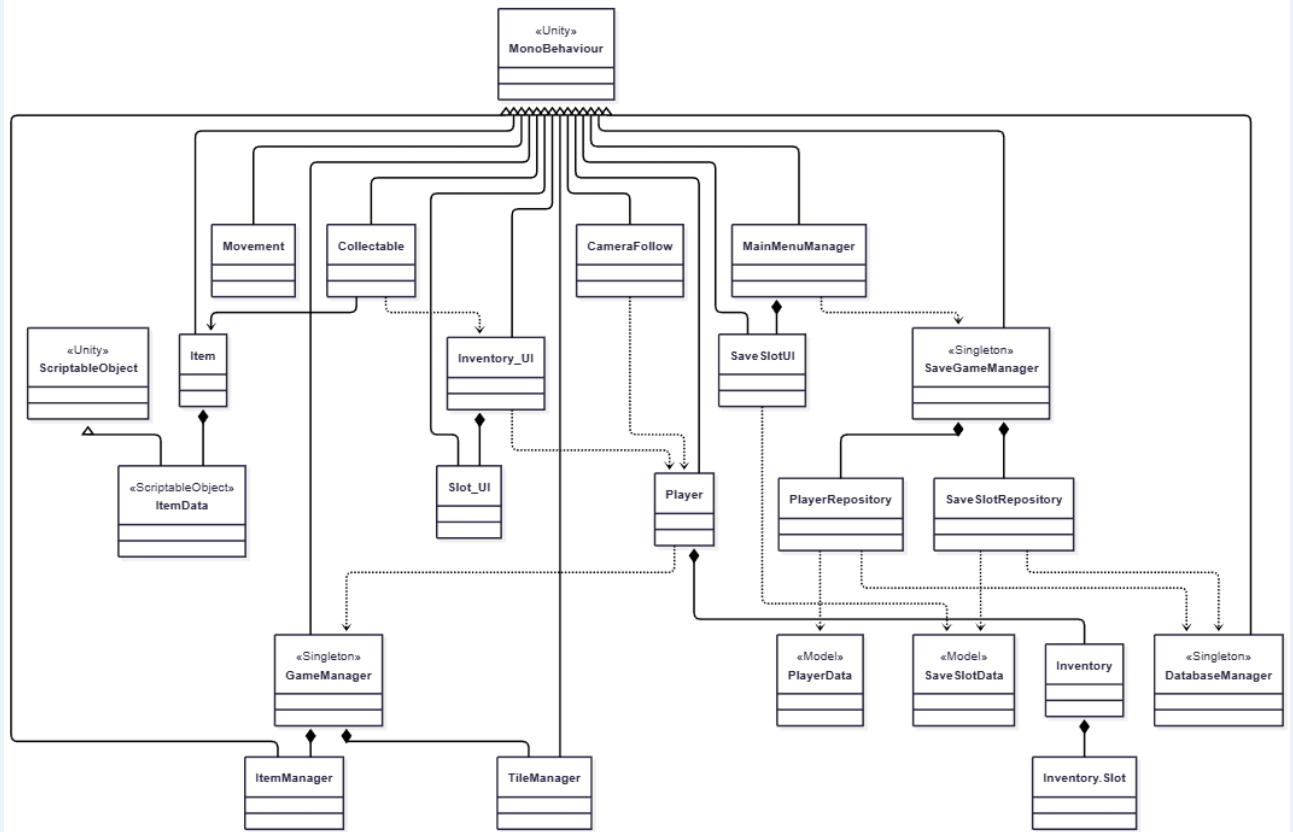


Diagrama desglosado: Gameplay

Diagrama: (disponible en Anexo I)

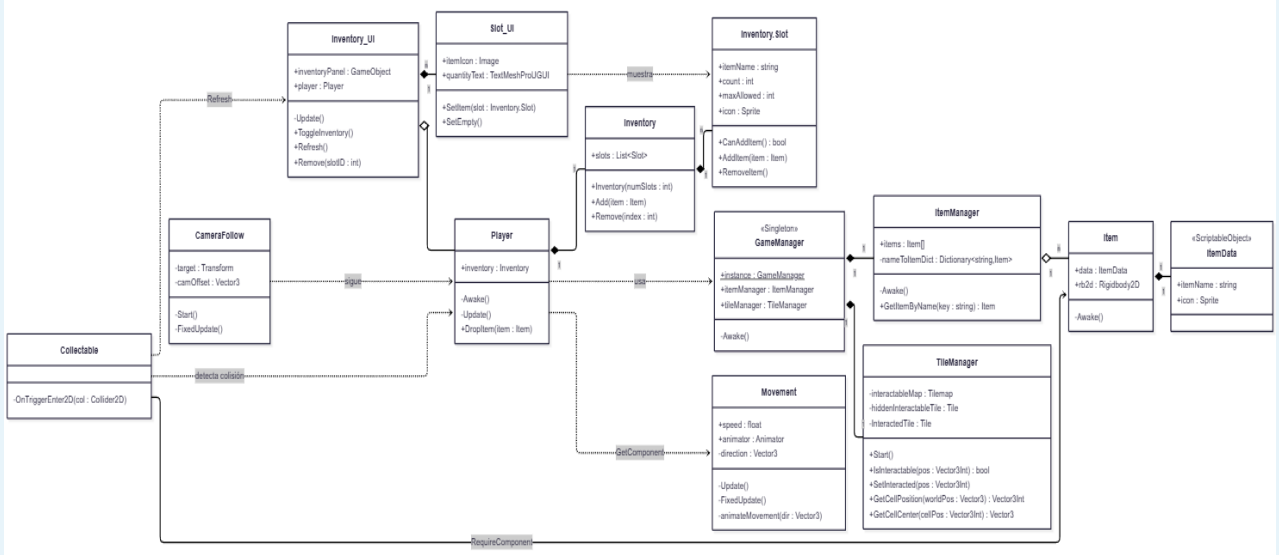


Diagrama desglosado: Saving System

Diagrama:

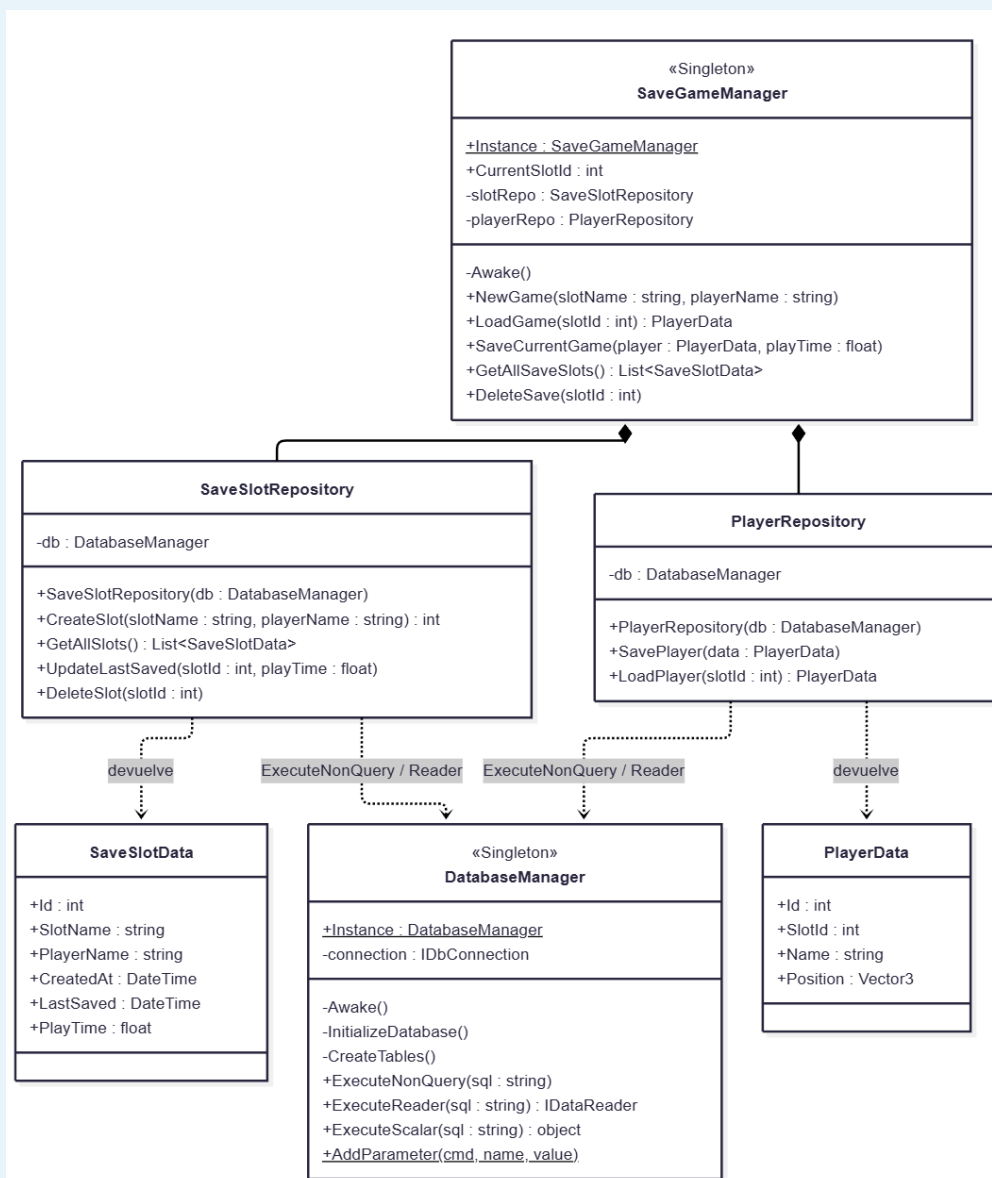
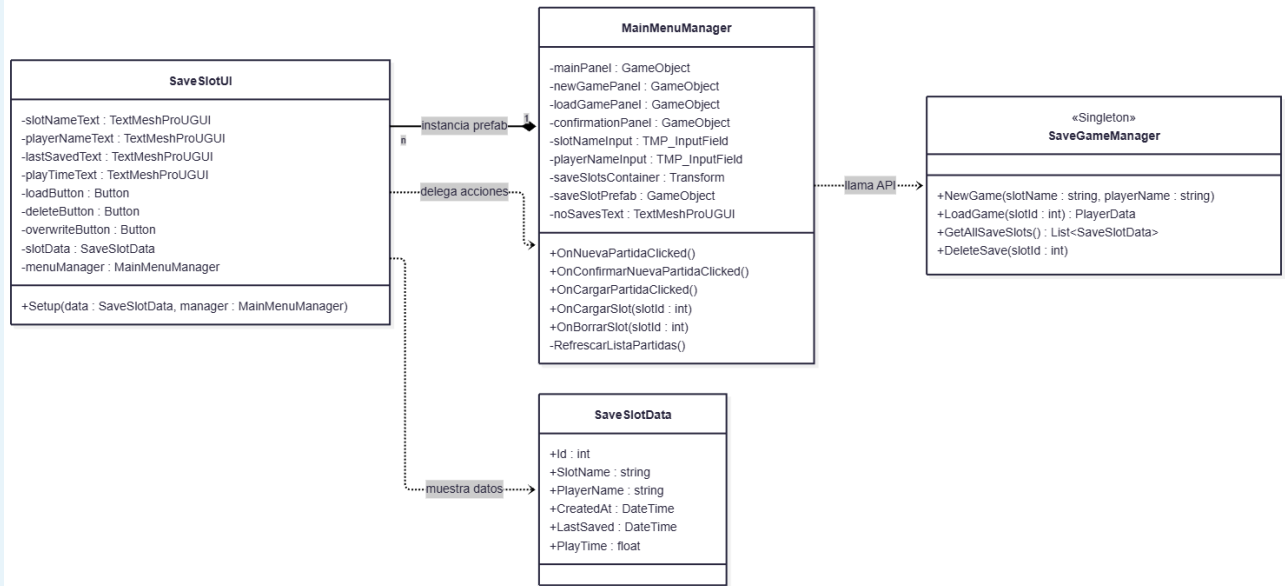


Diagrama desglosado: Main Menu

Diagrama:



Diagramas de secuencia

Diagrama: Nueva Partida

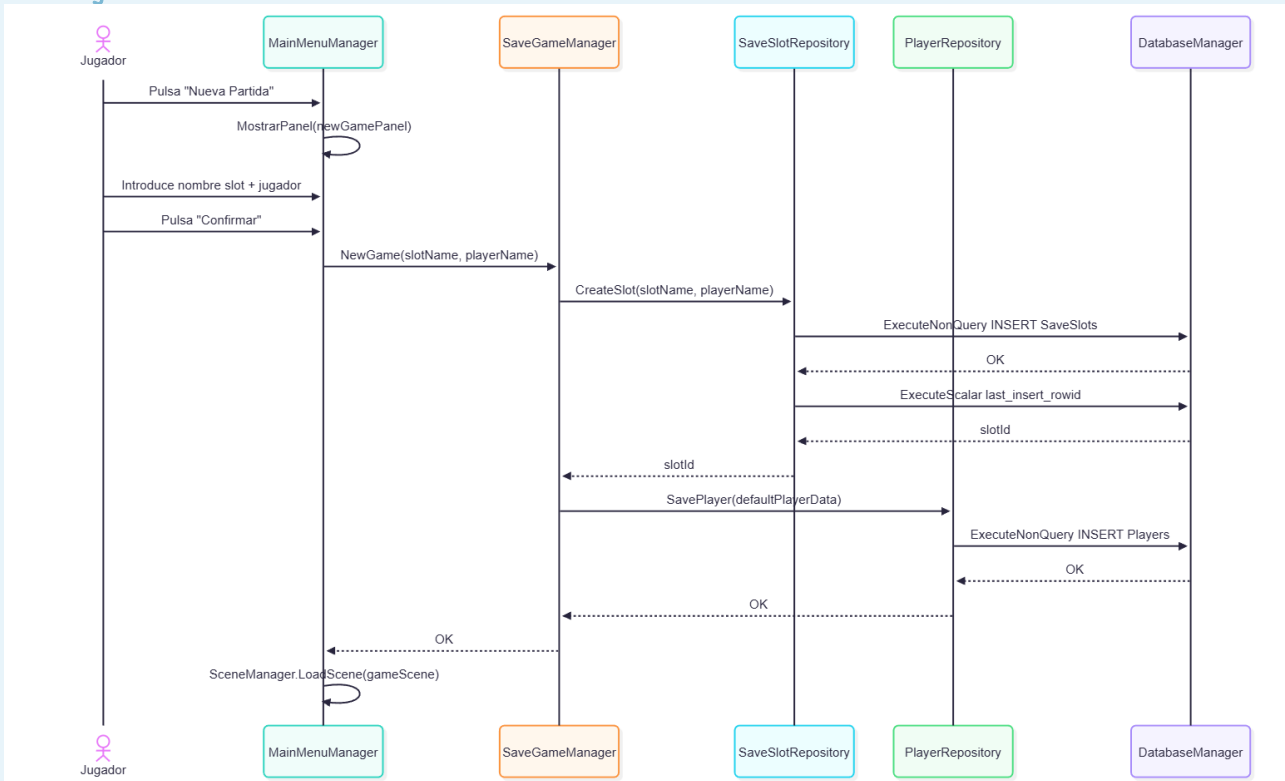


Diagrama: Cargar partida

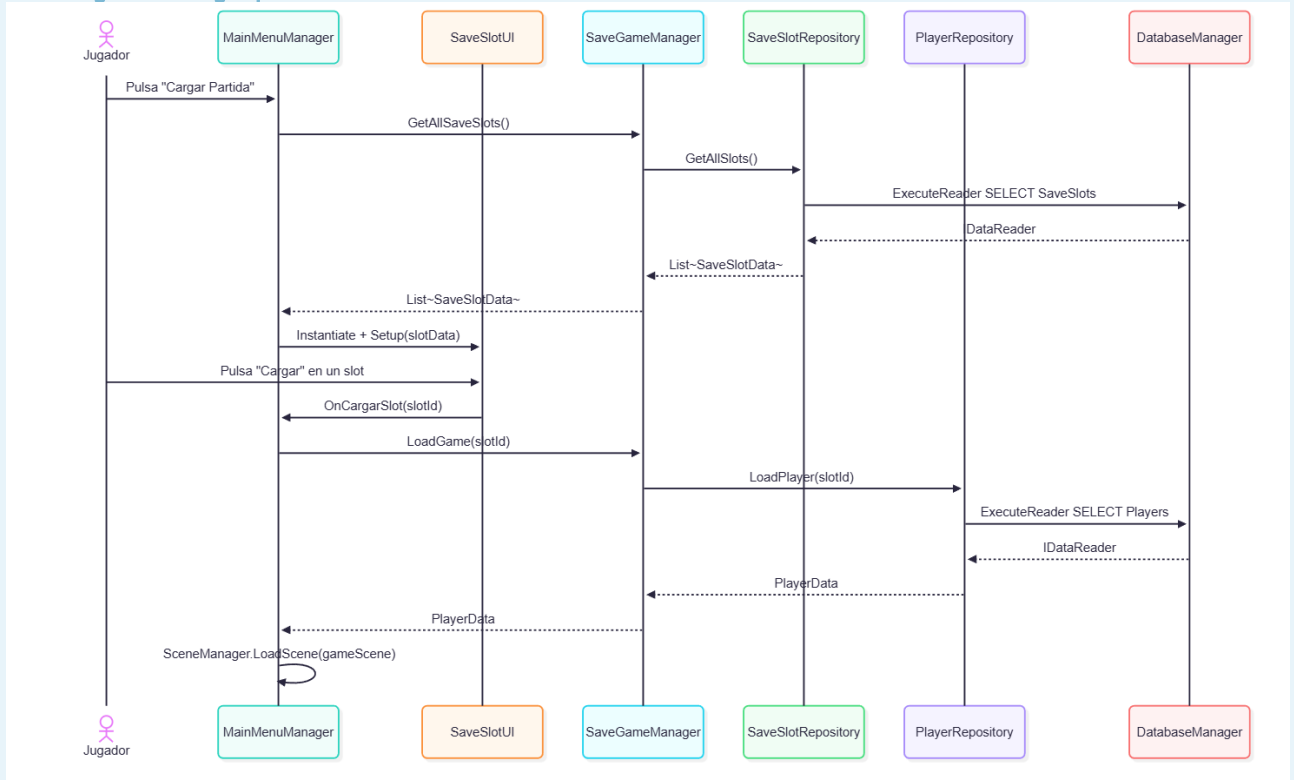
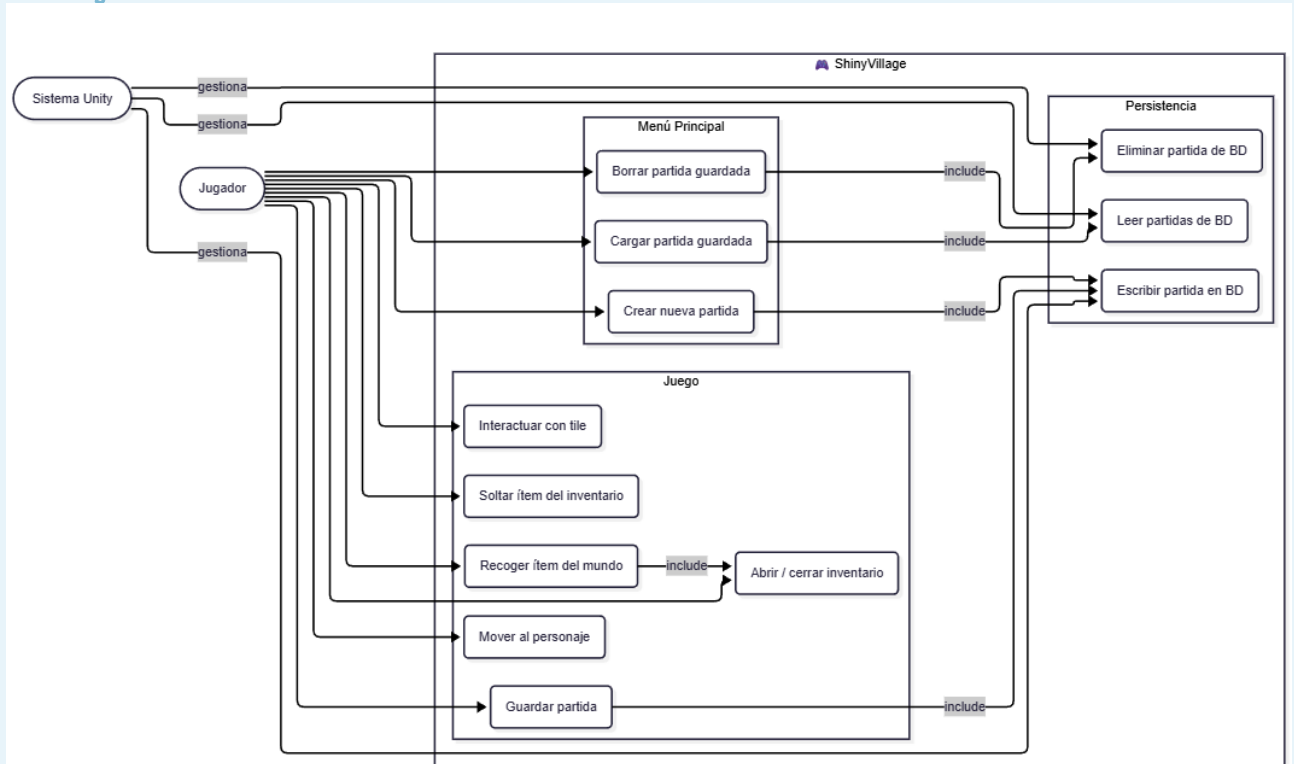
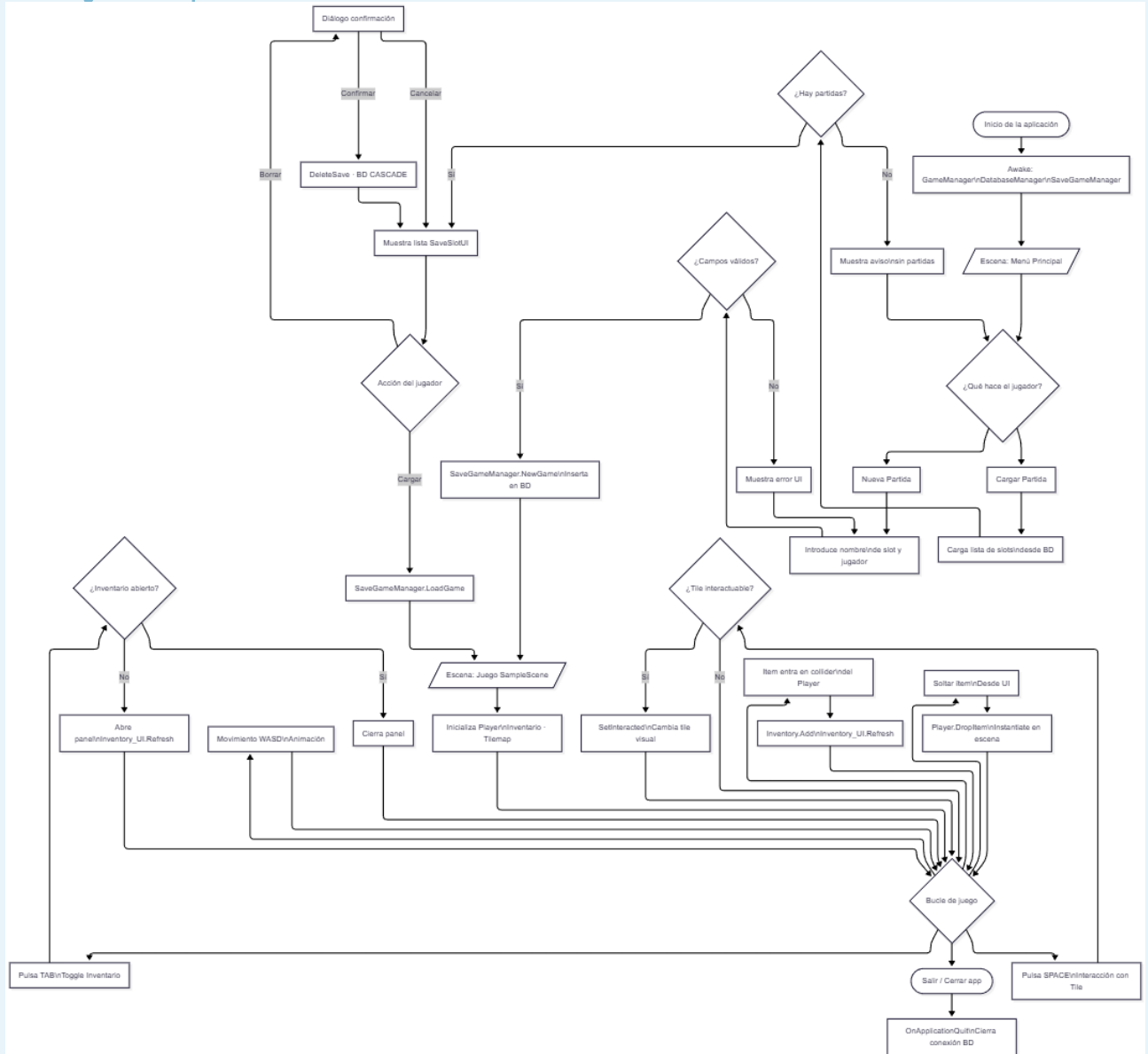


Diagrama de casos de uso

Diagrama:

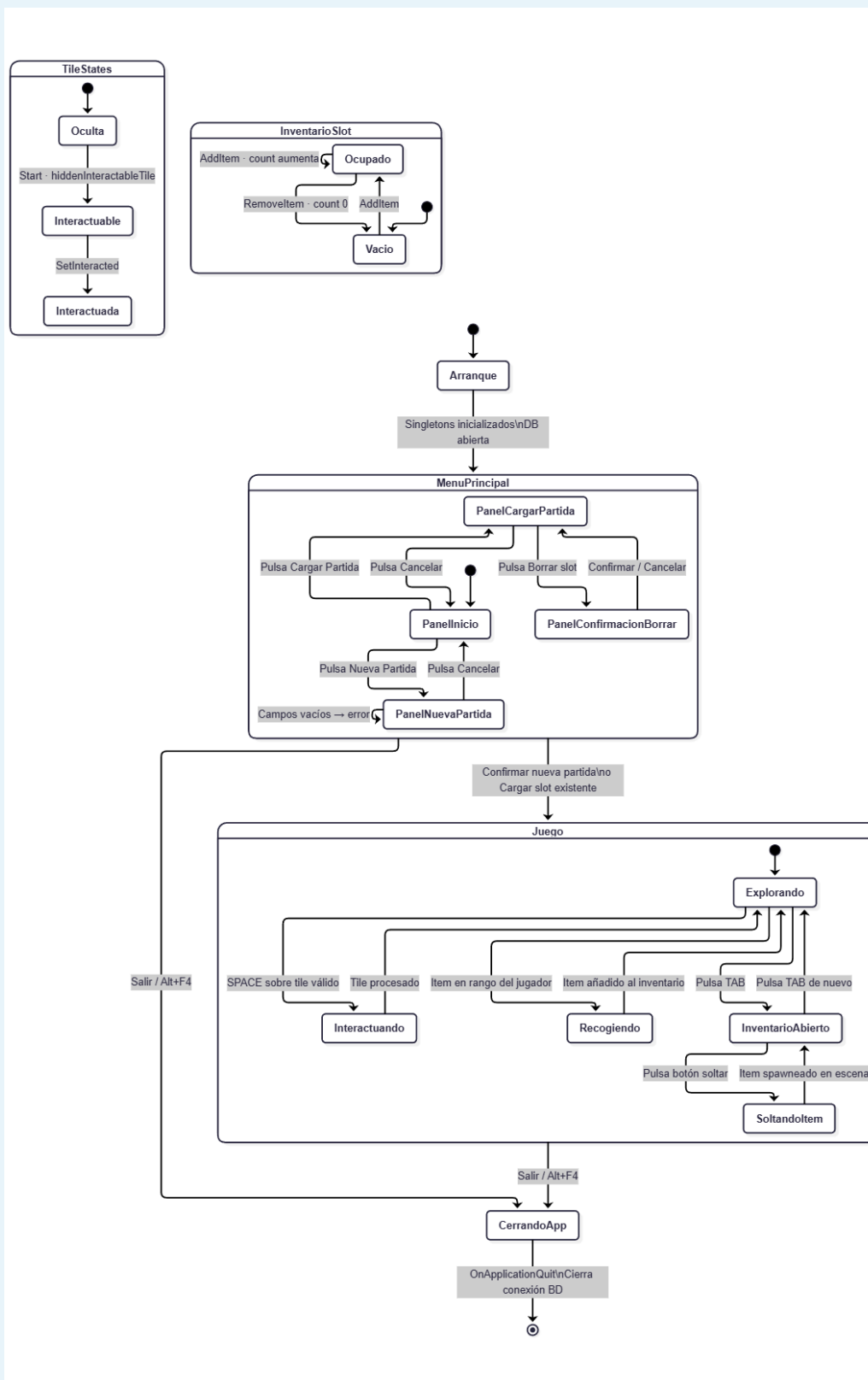


3.2.2 Diagrama de flujo

Diagrama: (Disponible en [Anexo I](#))

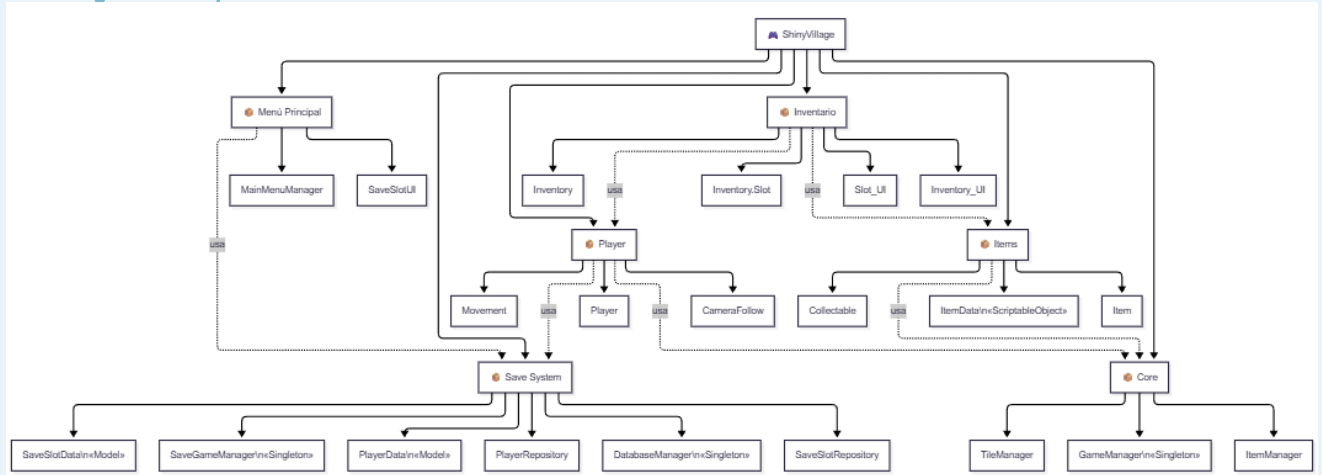
3.2.3 Diagrama de estados

Diagrama:



3.2.4 Descomposición en módulos

Diagrama: (Disponible en [Anexo I](#))



3.3 Análisis y diseño de la interfaz

3.3.1 Mockups

Para el diseño de los mockups se ha utilizado GoodNotes 6 en PadOS (IPad 10) debido a sus prestaciones como aplicación de notas y esbozos. En ella, se han realizado los mockups de las principales escenas y menús de la aplicación.

 **Mockups:** diagrams/Mockups.jpg — insertar imagen aquí

3.3.2 Diseño visual

El diseño visual utiliza sprites de estilo **Pixel Art** para una sensación retro y simplificada. El estilo con colores claros y en tonos pasteles le da un toque calmado y casual. El Asset Pack se obtiene desde [itch.io \(CupNooble\)](https://itch.io/CupNooble), que ofrece una versión gratuita muy completa.

Interfaz actual (WIP)

Pantalla	Descripción
Inventario — img/Inventory.png	Panel de gestión de ítems con slots visuales
Menú principal — img/mainmenu.png	Pantalla de inicio con acceso a partidas
Cargar partida — img/load.png	Lista de slots guardados con acciones
Interfaz de juego — img/tilemap.png	Vista cenital del mapa con Tilemap

 **Nota:** Las capturas están en modificación constante conforme avanza el desarrollo.

3.4 Análisis y diseño de la arquitectura

3.4.1 Tecnologías usadas y herramientas

Herramienta	Rol
Unity	Motor de juego principal
C#	Lenguaje para los scripts
Visual Studio Code	IDE
Git / GitHub	Control de versiones y ramas de trabajo
Mermaid / Markdown	Documentación técnica del repositorio y diagramas
SQLite	Base de datos de persistencia
IPad + GoodNotes	Diagramas, mockups y notas

Motor de Juego — Unity

Unity se ha seleccionado como motor de juego por ser un entorno completo para desarrollar RPGs 2D. Posee sistemas integrados para el control de Tilemap, física 2D y UI Canvas. La alternativa inicial fue Godot 4, aunque el sistema de assets y la lógica 2D no están tan integradas y resultó complicado aprender a manejarlas al inicio.

Lenguaje: C# / .NET Standard

C# es el único lenguaje de scripting que soporta Unity. Se ha elegido frente al Visual Scripting porque puede ser versionado con Git de forma muy limpia y permite utilizar patrones como Singleton o Repository de manera explícita. C# es uno de los lenguajes estándar en la industria del videojuego junto a C++.

Entorno de desarrollo — VS Code

Se ha elegido VS Code frente a JetBrains Rider principalmente porque es gratuito y tiene integración fluida con Git y GitHub. Posee extensiones específicas para este proyecto: GitLens, MarkdownAllInOne y la extensión oficial de Unity.

La estrategia de ramas utilizada es: main (rama estable que recibe merges de funcionalidades completas) y feature-X (rama por funcionalidad en desarrollo).

Tecnologías para la documentación

Toda la documentación técnica se escribe en Markdown por su formato de texto plano, versionable con Git y con renderizado nativo en GitHub. La memoria del proyecto se expone en la [Wiki del repositorio](#) mediante un Workflow que tras cada commit actualiza la wiki de forma automatizada.

Paquetes y dependencias

NuGetForUnity

NuGetForUnity es un gestor de paquetes NuGet integrado en el Editor de Unity. Se usa para instalar System.Data.SQLite. La alternativa habitual (DLLs manuales en Assets/Plugins) es propensa a errores de versión y difícil de mantener.

SQLite

SQLite se emplea como motor de base de datos para el sistema de partidas guardadas. Al ser una solución embebida, no requiere servidor externo. Es mucho más robusta que PlayerPrefs, que solo admite tipos primitivos y no sobrevive a reinstalaciones. La arquitectura de acceso sigue el patrón Repository (SaveSlotRepository, PlayerRepository), que separa la lógica de negocio del acceso a datos.

New Input System

El nuevo Input System de Unity se emplea en lugar del sistema legacy (Input.GetKey) por ser el único que recibirá mantenimiento a largo plazo. Permite mapear controles de forma flexible desde un asset centralizado (InputActions).

3.4.2 Arquitectura de los componentes

Estructura física: carpetas del proyecto

La estructura de carpetas sigue las convenciones estándar de Unity, con todo el código fuente bajo Assets/Scripts/ organizado por responsabilidad:

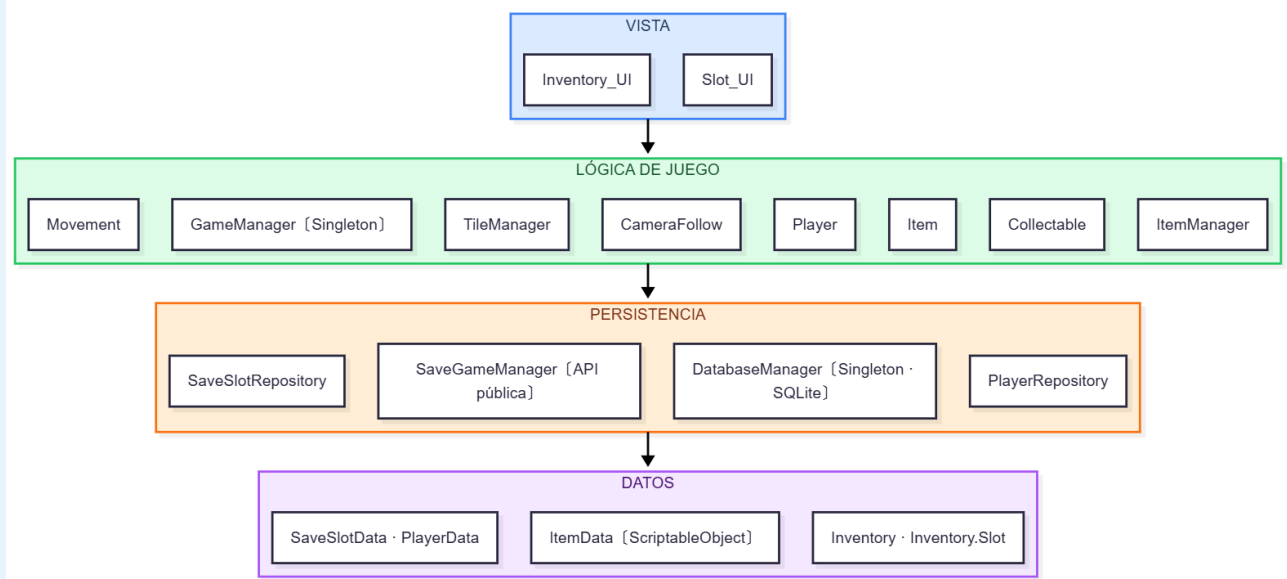
```
Assets/
├── Scripts/
│   ├── Database/
│   │   ├── DatabaseManager.cs
│   │   ├── SaveGameManager.cs
│   │   ├── SaveSlotRepository.cs
│   │   └── PlayerRepository.cs
│   ├── UI/
│   │   ├── Inventory_UI.cs
│   │   └── Slot_UI.cs
│   ├── ScriptableObject/
│   │   └── ItemData.cs
│   ├── Player.cs
│   ├── Movement.cs
│   ├── CameraFollow.cs
│   ├── Item.cs
│   ├── Collectable.cs
│   ├── Inventory.cs
│   ├── ItemManager.cs
│   ├── TileManager.cs
│   └── GameManager.cs
├── Scenes/
│   └── SampleScene.unity
├── Packages/          ← DLLs de NuGet (SQLite)
└── InputSystem_Actions.inputactions
```

Estructura lógica: capas y patrones

El proyecto implementa una arquitectura en capas equivalente a MVC: la regla de dependencia fluye siempre hacia abajo. Ninguna capa inferior conoce a las capas superiores.

Capa	Componentes principales	Responsabilidad
Presentación (Vista)	Inventory_UI, Slot_UI	Mostrar el estado en pantalla. Sin lógica de juego.
Lógica de juego (Controlador)	GameManager, Player, Movement, Collectable, TileManager, ItemManager	Comportamiento del juego. MonoBehaviour.
Persistencia (Repository)	DatabaseManager, SaveGameManager, SaveSlotRepository, PlayerRepository	Acceso a datos SQLite. Patrón Repository.
Datos (Modelo)	ItemData, Inventory, Inventory.Slot, SaveSlotData, PlayerData	Datos puros. Sin lógica de juego ni presentación.

Diagrama:



4. Implementación y pruebas

Este capítulo detalla la implementación técnica completa del proyecto ShinyVillage, organizado por subsistemas. Se presentan los fragmentos de código más relevantes con explicaciones detalladas de su funcionamiento, así como las pruebas realizadas para verificar el correcto comportamiento de cada componente.

4.1 Implementación

4.1.1 Sistema de Movimiento y Controles

El sistema de movimiento del jugador constituye la base de la interacción en el juego. Se implementó un sistema de movimiento en 8 direcciones con animaciones fluidas y física 2D.

Clase Movement – Movement.cs

El script Movement gestiona el desplazamiento del personaje y la sincronización con el sistema de animaciones:

```
public class Movement : MonoBehaviour
{
    public float speed;
    public Animator animator;
    private Vector3 direction;

    private void Update()
    {
        // Se obtiene el input del jugador
        float horizontal = Input.GetAxisRaw("Horizontal");
        float vertical = Input.GetAxisRaw("Vertical");

        // Se crea el vector de dirección
        direction = new Vector3(horizontal, vertical);
        direction = direction.normalized; // Normaliza para velocidad constante

        animateMovement(direction);
    }

    // Se usa FixedUpdate para física consistente
    private void FixedUpdate()
    {
        transform.position += direction * speed * Time.deltaTime;
    }

    void animateMovement(Vector3 direction)
    {
        if (animator != null)
        {

```

```
        if (direction.magnitude > 0) // Hay movimiento
        {
            animator.SetBool("isMoving", true);
            animator.SetFloat("horizontal", direction.x);
            animator.SetFloat("vertical", direction.y);
        }
        else
        {
            animator.SetBool("isMoving", false);
        }
    }
}
```

Explicación técnica:

- Se utiliza `GetAxisRaw` en lugar de `GetAxis` para obtener valores discretos (-1, 0, 1) que proporcionan un control más preciso.
- La normalización del vector dirección (`direction.normalized`) garantiza que el movimiento diagonal tenga la misma velocidad que el movimiento en los ejes cardinales, evitando que el jugador se mueva más rápido en diagonal.
- Se separa la lectura del input (`Update`) del cálculo de física (`FixedUpdate`). Esto previene el efecto de 'rebote' que ocurriría si el movimiento se calculara antes que el collider en cada frame.
- `Time.deltaTime` asegura que el movimiento sea independiente de la tasa de frames, manteniendo velocidad consistente en diferentes hardware.
- El Animator recibe tanto el estado de movimiento (`isMoving`) como la dirección (`horizontal`, `vertical`) para seleccionar la animación apropiada mediante un Blend Tree.

Clase CameraFollow - CameraFollow.cs

El script CameraFollow implementa un seguimiento suave del jugador manteniendo un offset constante:

```
public class CameraFollow : MonoBehaviour
{
    [SerializeField] private Transform target;
    Vector3 camOffset;

    void Start()
    {
        // Se calcula el offset inicial
        camOffset = transform.position - target.position;
    }

    private void FixedUpdate()
    {
        transform.position = target.position + camOffset;
    }
}
```

Explicación técnica:

- El SerializeField permite editar la referencia al jugador en el Inspector manteniendo el campo privado, siguiendo el principio de encapsulación.
- Se usa FixedUpdate en lugar de LateUpdate para sincronizar con el movimiento del jugador, que también se actualiza en FixedUpdate, evitando así cualquier jittering visual.
- El offset se calcula una vez en Start y se reutiliza, lo que es más eficiente que recalcularlo cada frame.

4.1.2 Sistema de Inventario

El sistema de inventario permite al jugador recolectar, almacenar y gestionar objetos. Se implementó utilizando una arquitectura modular que separa la lógica de datos (Inventory) de la presentación (Inventory_UI).

Clase Inventory – Inventory.cs

La clase Inventory gestiona la estructura de datos del inventario y la lógica de apilamiento de objetos:

```
[System.Serializable]
public class Inventory
{
    [System.Serializable]
    public class Slot
    {
        public string itemName;
        public int count;
        public int maxAllowed;
        public Sprite icon;

        public bool CanAddItem()
        {
            return count < maxAllowed;
        }

        public void AddItem(Item item)
        {
            this.itemName = item.data.itemName;
            this.icon = item.data.icon;
            count++;
        }

        public void RemoveItem()
        {
            if (count > 0)
            {
                count--;
                if (count == 0)
                {
                    icon = null;
                    itemName = "";
                }
            }
        }
    }
}
```



```
        }  
    }  
}  
  
public List<Slot> slots = new List<Slot>();  
  
// Lógica de añadir items con apilamiento automático  
public void Add(Item item)  
{  
    // Busca si ya existe el item y tiene espacio  
    foreach (Slot slot in slots)  
    {  
        if (slot.itemName == item.data.itemName && slot.CanAddItem())  
        {  
            slot.AddItem(item);  
            return;  
        }  
    }  
  
    // Si no existe, busca un slot vacío  
    foreach (Slot slot in slots)  
    {  
        if (slot.itemName == "")  
        {  
            slot.AddItem(item);  
            return;  
        }  
    }  
}
```

Explicación técnica:

- El atributo [System.Serializable] permite que Unity serialice la clase para el Inspector y el sistema de guardado, fundamental para la persistencia de datos.
- El sistema implementa apilamiento inteligente: primero busca slots existentes del mismo tipo con espacio disponible, luego busca slots vacíos. Esto optimiza el uso del espacio del inventario.
- El método CanAddItem() encapsula la lógica de verificación de capacidad, facilitando cambios futuros en las reglas de apilamiento.
- RemoveItem() limpia completamente el slot cuando count llega a 0, liberando referencias y evitando estados inconsistentes.

Método DropItem – Player.cs

El método DropItem crea un efecto visual al soltar objetos del inventario:

```
public void DropItem(Item item)
{
    Vector2 spawnLocation = transform.position;

    // Offset aleatorio para evitar recogida inmediata
    Vector2 spawnOffset = Random.insideUnitCircle * 2f;

    // Se instancia el objeto en el mundo
    Item droppedItem = Instantiate(item, spawnLocation + spawnOffset,
    Quaternion.identity);

    // Se añade fuerza para efecto visual
    droppedItem.rb2d.AddForce(spawnOffset * 0.2f, ForceMode2D.Impulse);
}
```

Explicación técnica:

- Random.insideUnitCircle genera un vector aleatorio dentro de un círculo unitario, creando variedad visual al soltar múltiples objetos.
- El offset de 2 unidades garantiza que el objeto aparezca fuera del collider del jugador, previniendo que se recoja automáticamente al instante.
- AddForce con ForceMode2D.Impulse aplica una fuerza instantánea, creando un efecto de 'lanzamiento' suave que mejora la retroalimentación visual.

4.1.3 Sistema de Tiles y Mapa Interactivo

El sistema de tiles gestiona el mapa del juego utilizando el Tilemap de Unity, permitiendo la interacción del jugador con elementos del entorno.

Clase TileManager – TileManager.cs (fragmento)

```
public class TileManager : MonoBehaviour
{
    private Tilemap interactableMap;
    private Tile hiddenInteractableTile;
    private Tile InteractedTile;

    // Verifica si una posición es interactuable
    public bool IsInteractable(Vector3Int pos)
    {
        TileBase tile = interactableMap.GetTile(pos);
        return tile == hiddenInteractableTile;
    }

    // Marca un tile como interactuado
    public void SetInteracted(Vector3Int pos)
    {

```

```
        interactableMap.SetTile(pos, InteractedTile);
    }

    // Convierte posición del mundo a coordenadas de celda
    public Vector3Int GetCellPosition(Vector3 worldPos)
    {
        return interactableMap.WorldToCell(worldPos);
    }
}
```

Explicación técnica:

- El sistema utiliza dos tipos de tiles diferentes: `hiddenInteractableTile` (invisible) para marcar posiciones interactivables, e `InteractedTile` (visible) para mostrar el resultado de la interacción.
- `WorldToCell()` convierte coordenadas del mundo (continuas) a coordenadas de grid (discretas), permitiendo mapear la posición del jugador a celdas específicas del `tilemap`.
- `SetTile()` modifica el `Tilemap` en tiempo de ejecución, permitiendo feedback visual inmediato de las acciones del jugador.

Lógica de Interacción – `Player.cs`

El jugador puede interactuar con tiles adyacentes basándose en su dirección de movimiento:

```
void Update()
{
    if (Input.GetKeyDown(KeyCode.Space))
    {
        Movement movement = GetComponent<Movement>();
        // Se obtiene la última dirección del animator
        Vector3 lastDir = new Vector3(
            movement.animator.GetFloat("horizontal"),
            movement.animator.GetFloat("vertical")
        ).normalized;

        // Dirección por defecto si está parado
        if (lastDir == Vector3.zero)
            lastDir = Vector3.down;

        // Convierte a dirección cardinal
        Vector3Int cardinalDir;
        if (Mathf.Abs(lastDir.x) > Mathf.Abs(lastDir.y))
            cardinalDir = new Vector3Int((int)Mathf.Sign(lastDir.x), 0, 0);
        else
            cardinalDir = new Vector3Int(0, (int)Mathf.Sign(lastDir.y), 0);

        // Calcula la celda objetivo
        Vector3Int playerCell =
            GameManager.instance.tileManager.GetCellPosition(transform.position);
        Vector3Int targetCell = playerCell + cardinalDir;
    }
}
```

```
// Interactúa si es posible
if (GameManager.instance.tileManager.IsInteractable(targetCell))
{
    GameManager.instance.tileManager.SetInteracted(targetCell);
}
}
```

Explicación técnica:

- Se recupera la última dirección del Animator en lugar de calcularla nuevamente, garantizando coherencia con la animación visual del personaje.
- La conversión a dirección cardinal (solo horizontal o vertical, nunca diagonal) simplifica la interacción con el grid cuadrado del Tilemap.
- `Mathf.Sign` devuelve -1, 0 o 1, permitiendo conversión directa a dirección cardinal.
- El uso del Singleton `GameManager.instance` permite acceso centralizado al `TileManager` sin necesidad de referencias duplicadas.

4.1.1 Sistema de Base de Datos

El sistema de base de datos constituye uno de los componentes más críticos del proyecto, ya que gestiona la persistencia de todas las partidas guardadas. Se implementó utilizando SQLite como motor de base de datos por su naturaleza embebida y su capacidad para funcionar sin necesidad de un servidor externo.

Método `ExecuteQuery` – `DatabaseManager.cs`

Este método es fundamental para la lectura de datos de la base de datos. A diferencia del enfoque tradicional con `ExecuteReader` que mantiene la conexión abierta mientras se leen los datos, esta implementación extrae todos los resultados inmediatamente y los almacena en memoria, liberando la conexión de inmediato:

```
// Devuelve los datos ya leídos en una lista, en lugar de devolver un reader "vivo".
// Así el comando se cierra inmediatamente y la conexión queda libre.
public List<Dictionary<string, object>> ExecuteQuery(string sql,
SqlParameterizer parameterize = null)
{
    var results = new List<Dictionary<string, object>>();

    // using → se libera siempre
    using (IDbCommand cmd = _connection.CreateCommand())
    {
        cmd.CommandText = sql;
        parameterize?.Invoke(cmd);

        // using → reader se cierra siempre
        using (IDataReader reader = cmd.ExecuteReader())
        {
            while (reader.Read())
```

```
{
    var row = new Dictionary<string, object>();
    // Guardamos columna por nombre
    for (int i = 0; i < reader.FieldCount; i++)
        row[reader.GetName(i)] = reader.GetValue(i);
    results.Add(row);
}
} // reader se cierra aquí
} // comando se libera aquí

return results; // devolvemos datos ya extraídos
}
```

Explicación técnica:

- Se utiliza el patrón 'using' para garantizar la liberación automática de recursos (conexión y reader) incluso si ocurre una excepción.
- Los resultados se almacenan en un Dictionary<string, object> donde la clave es el nombre de la columna. Esto hace que el acceso a los datos sea más legible y menos propenso a errores que usar índices numéricos.
- Se evita el problema de mantener la conexión ocupada, lo cual podría causar bloqueos en operaciones subsecuentes.
- El delegate SqlParameterizer permite inyectar parámetros de forma segura, previniendo inyecciones SQL.

Método AddParameter – DatabaseManager.cs

Este método estático proporciona una forma segura de añadir parámetros a las consultas SQL, evitando inyecciones SQL y problemas con caracteres especiales:

```
/// Ejemplo:
/// DatabaseManager.AddParameter(cmd, "@player_name", "O'Brien"); // funciona
/// cmd.CommandText = "... WHERE name = 'O'Brien'"; // falla
public static IDbDataParameter AddParameter(IDbCommand cmd, string name,
object value)
{
    var param = cmd.CreateParameter();
    param.ParameterName = name;

    // Manejo especial de valores nulos
    if (value == null)
        param.Value = System.DBNull.Value;
    else
        param.Value = value;

    cmd.Parameters.Add(param);
    return param;
}
```

Explicación técnica:

- El método convierte valores null de C# a DBNull.Value de SQL, que es el tipo correcto para representar valores nulos en base de datos.
- Se utiliza el namespace completo System.DBNull para evitar ambigüedades con otras clases.
- El uso de parámetros en lugar de concatenación de strings previene inyecciones SQL. Por ejemplo, un nombre como "O'Brien" contiene una comilla simple que rompería una consulta SQL construida con concatenación.

4.1.2 Sistema de Islas Procedurales

Se implementó un sistema que genera colores únicos para cada partida guardada, de modo que cada isla (partida) tenga su propia identidad visual. Este sistema utiliza generación procedural basada en semillas para garantizar consistencia entre cargas. Es el sistema mas sencillo que se ha encontrado para representar los "viajes" entre islas de los diferentes slots de guardado.

Clase IslandGenerator - IslandGenerator.cs

Esta clase estática genera configuraciones visuales únicas para cada isla basándose en el ID del slot de guardado:

```
/// Genera una configuración de isla basada en un ID de slot.
/// Usa el ID como semilla para generar colores consistentes.
public static IslandConfig GenerateIsland(int slotId, string islandName)
{
    // Se usa el slotId como semilla para que siempre genere los mismos
    colores

    // Multiplicamos por 1000 para mayor variación de los colores
    Random.InitState(slotId * 1000);

    // Se crea la configuración en memoria (no como asset en disco)
    IslandConfig config = ScriptableObject.CreateInstance<IslandConfig>();
    config.slotId = slotId;
    config.islandName = islandName;

    // Generación de colores usando rangos específicos
    configgroundColor = GenerateGroundColor(); // verdes, marrones,
    amarillos
    config.waterColor = GenerateWaterColor(); // azules, turquesas
    config.accentColor = GenerateAccentColor(); // colores vibrantes
    config.globalTint = GenerateGlobalTint(); // tinte sutil

    // Se restaura el estado aleatorio para no afectar otros sistemas
    Random.InitState(System.DateTime.Now.Millisecond);

    return config;
}
```

Explicación técnica:

- La semilla se calcula multiplicando el slotId por 1000, lo que proporciona una mejor distribución de los números aleatorios y evita que IDs consecutivos generen paletas similares.
- Se utiliza ScriptableObject.CreateInstance para crear la configuración en memoria en lugar de crear un asset en disco. Esto evita ensuciar el proyecto con archivos generados dinámicamente.
- Al finalizar se restaura el estado del generador de números aleatorios para que otros sistemas del juego no se vean afectados por esta semilla específica.
- Los colores se generan usando el espacio HSV (Hue, Saturation, Value) en lugar de RGB, lo que permite un mayor control sobre la armonía cromática y garantiza que los colores sean visualmente agradables.

Generación de Colores de Terreno

```
private static Color GenerateGroundColor()
{
    // Hue entre 30-120 grados = amarillos, verdes, marrones
    float hue = Random.Range(0.08f, 0.35f);
    // Saturación media para colores naturales
    float saturation = Random.Range(0.3f, 0.6f);
    // Brillo medio
    float value = Random.Range(0.5f, 0.8f);

    return Color.HSVToRGB(hue, saturation, value);
}
```

Los valores de Hue se mapean de 0 a 1 en Unity (donde 0 y 1 son rojo, 0.33 es verde y 0.66 es azul). El rango 0.08-0.35 selecciona tonos naturales de tierra: amarillos, verdes oliva y marrones, evitando colores poco realistas como rosas o azules para el terreno.

Clase IslandManager – IslandManager.cs

Este manager actúa como puente entre el sistema de guardado y la visualización, aplicando la configuración de colores al Tilemap:

```
public void LoadIsland(SaveSlotData slotData)
{
    if (slotData == null)
    {
        Debug.LogError("[IslandManager] Datos NULOS");
        return;
    }

    // Se genera la configuración usando la semilla guardada
    _currentIsland = IslandGenerator.GenerateIsland(
        slotData.IslandSeed, slotData.SlotName
    );

    // Se aplica la paleta de colores al Tilemap
    ApplyIslandVisuals();
}
```

El método `ApplyIslandVisuals()` utiliza la propiedad `Tilemap.color` para aplicar un tinte global a todos los tiles de forma eficiente, sin necesidad de modificar cada tile individualmente.

4.1.3 Sistema de Menú Principal

Se desarrolló un sistema completo de menú con navegación entre paneles, gestión de partidas guardadas y validación de entrada de usuario.

Clase `MainMenuManager` – `MainMenuManager.cs`

La gestión de la lista de partidas guardadas presenta un desafío técnico interesante: Unity necesita un frame completo para activar un panel y procesar su layout antes de poder instanciar elementos dentro de él. La solución implementada utiliza una corrutina:

```
public void OnCargarPartidaClickado()
{
    MostrarPanel(loadGamePanel);

    // Esperamos 1 frame para que Unity active el panel y procese
    // su RectTransform antes de instanciar los items
    StartCoroutine(RefrescarListaPartidasDelayed());
}

private IEnumerator RefrescarListaPartidasDelayed()
{
    yield return null; // espera 1 frame
    RefrescarListaPartidas();
}
```

Explicación técnica:

- `yield return null` hace que espere exactamente un frame antes de continuar. Esto da tiempo a Unity para procesar la activación del panel y calcular sus dimensiones.
- Sin este tiempo, el `ContentSizeFitter` del `ScrollView` calcularía tamaños incorrectos porque intentaría medir el contenido antes de que el panel esté completamente inicializado (sucedió al principio y fue la causa de muchos errores de interfaz posteriores)

Gestión Dinámica de Layout

Otro problema encontrado fue que el `VerticalLayoutGroup` no recalculaba correctamente la altura total del `ScrollView` al instanciar elementos dinámicamente. La solución requiere forzar el recalclo del layout:

```
// Instanciamos el prefab de la partida guardada
GameObject slotGO = Instantiate(saveSlotPrefab, saveSlotsContainer);

// Forzamos la altura por código
LayoutElement le = slotGO.GetComponent<LayoutElement>();
if (le == null)
    le = slotGO.AddComponent<LayoutElement>();

le.preferredHeight = 120f; // altura fija de cada fila
```



```
// ... (instanciar todos los elementos)

// Forzamos recálculo del layout
RectTransform contentRT = (RectTransform) saveSlotsContainer;
Canvas.ForceUpdateCanvases();
LayoutRebuilder.ForceRebuildLayoutImmediate(contentRT);
```

Explicación técnica:

- `LayoutElement.preferredHeight` indica al sistema de layout cuánto espacio necesita cada elemento. Sin esto, el `ContentSizeFitter` no puede calcular la altura total correctamente.
- `Canvas.ForceUpdateCanvases()` fuerza la actualización inmediata de todos los Canvas en la jerarquía.
- `LayoutRebuilder.ForceRebuildLayoutImmediate()` reconstruye el layout del elemento especificado en el mismo frame, en lugar de esperar al siguiente frame.

Componente `SaveSlot_UI` - `SaveSlot_UI.cs`

Cada elemento de la lista de partidas es un prefab instanciado dinámicamente que muestra la información de una partida y proporciona botones de acción. El método `Setup` configura el componente con los datos correspondientes:

```
public void Setup(SaveSlotData data, MainMenuManager manager)
{
    _slotData = data;
    _menuManager = manager;

    // Rellenar textos
    slotNameText.text = data.SlotName;
    playerNameText.text = $"Player: {data.PlayerName}";
    lastSavedText.text = $"Last saved: {data.LastSaved:dd/MM/yyyy HH:mm}";
    playTimeText.text = FormatPlayTime(data.PlayTime);

    // Configurar botones
    // Limpieza defensiva para evitar listeners duplicados
    loadButton.onClick.RemoveAllListeners();
    deleteButton.onClick.RemoveAllListeners();
    overwriteButton.onClick.RemoveAllListeners();

    // Añadir listeners con lambdas
    loadButton.onClick.AddListener(
        () => _menuManager.OnLoadSlotClicked(_slotData.Id)
    );
    deleteButton.onClick.AddListener(
        () => _menuManager.OnDeleteSlotClicked(_slotData.Id, gameObject)
    );
}
```

Explicación técnica:

- Se utiliza string interpolation (\$) para formatear los textos de forma legible. La sintaxis {data.LastSaved:dd/MM/yyyy HH:mm} aplica formato de fecha directamente en la interpolación.
- RemoveAllListeners() es crucial para evitar que se acumulen listeners duplicados cuando se refresca la lista. Sin esto, cada refresco añadiría nuevos listeners a los existentes, causando que las acciones se ejecuten múltiples veces.
- Las expresiones lambda () => capturelran el valor actual de _slotData.Id, permitiendo que cada botón sepa qué partida debe cargar/borrar.

4.1.4 Código de Terceros Utilizado

Durante el desarrollo se han utilizado las siguientes bibliotecas y paquetes de terceros:

System.Data.SQLite

- Justificación: Proporciona la implementación de SQLite para .NET, permitiendo el uso de bases de datos embebidas sin dependencias externas.
- Licencia: Dominio público (Public Domain).
- Uso: Se utiliza a través de las interfaces estándar de System.Data (IDbConnection, IDbCommand, IDataReader), lo que facilita el cambio a otro motor de base de datos en el futuro si fuera necesario.

Sprout Lands Asset Pack

- Justificación: Pack de sprites que proporciona todos los elementos visuales del juego (tiles, personajes, objetos) con un estilo artístico coherente.
- Autor: Cup Nooble.
- Licencia: Gratuito para uso personal y comercial.
- Uso: Sprites del mapa, personaje jugador, objetos recolectables y elementos de interfaz.

4.2 Pruebas

Se han realizado pruebas exhaustivas de cada subsistema del proyecto para garantizar su correcto funcionamiento. Las pruebas se organizan por áreas funcionales.

4.2.1 Pruebas del Sistema de Movimiento

Prueba 1: Velocidad Constante en Diagonales

- Objetivo: Verificar que el movimiento diagonal tiene la misma velocidad que el movimiento cardinal.
- Procedimiento: Se midió la distancia recorrida en 5 segundos moviéndose en dirección cardinal y diagonal porque al inicio daba la sensación de que el movimiento en las diagonales era mucho más rápido.
- Resultado esperado: Ambas distancias deben ser iguales
- Resultado obtenido: Tras la normalización del vector, la prueba dio como resultado: Cardinal: 20.0 unidades. Diagonal: 20.02 unidades. La normalización del vector funciona correctamente.

Prueba 2: Sincronización de Animaciones

- Objetivo: Verificar que las animaciones corresponden con la dirección de movimiento.
- Procedimiento: Se observaron las animaciones mientras se movía el personaje en las 8 direcciones.
- Resultado esperado: El sprite debe mostrar la animación apropiada para cada dirección.
- Resultado obtenido: El Blend Tree del Animator selecciona correctamente la animación basándose en los valores horizontal y vertical.

Prueba 3: Seguimiento de Cámara

- Objetivo: Verificar que la cámara sigue suavemente al jugador sin movimientos extraños
- Procedimiento: Se movió el personaje rápidamente en varias direcciones observando el comportamiento de la cámara.
- Resultado esperado: La cámara debe seguir al jugador manteniendo un offset constante sin temblores.
- Resultado obtenido: El uso de FixedUpdate en ambos scripts (Movement y CameraFollow) garantiza sincronización perfecta.

4.2.2 Pruebas del Sistema de Inventario

Prueba 4: Apilamiento Automático

- Objetivo: Verificar que objetos del mismo tipo se apilan correctamente.
- Procedimiento: Se recogieron 5 semillas de cebolla consecutivamente.
- Resultado esperado: Todas las semillas deben ocupar un solo slot mostrando '5' como cantidad.
- Resultado obtenido: Tras algunas correcciones El método Add() encuentra el slot existente y incrementa count.

Prueba 5: Límite de Apilamiento

- Objetivo: Verificar que al llegar al límite (maxAllowed = 99) se crea un nuevo slot.
- Procedimiento: Se modificó temporalmente maxAllowed en el inspector a 3 y se recogieron 5 objetos iguales.
- Resultado esperado: Debe crear dos slots: uno con 3 objetos y otro con 2.
- Resultado obtenido: CanAddItem() retorna false cuando count == maxAllowed, forzando la creación de un nuevo slot.

Prueba 6: Soltar Objetos con retardo para que se no se puedan coger inmediatamente

- Objetivo: Verificar que los objetos soltados no se recogen inmediatamente.
- Procedimiento: Se recogió un objeto, se abrió el inventario y se soltó inmediatamente.
- Resultado esperado: El objeto debe aparecer cerca del jugador pero sin recogerse automáticamente.
- Resultado obtenido: Se corrigió añadiendo al soltar el objeto del inventario (que se hacía justo en la posición del jugador) un offset de Random.insideUnitCircle * 2f que así coloca el objeto fuera del trigger del jugador.

Prueba 7: Actualización de UI

- Objetivo: Verificar que la UI se actualiza correctamente al recoger y soltar objetos.
- Procedimiento: Se recogieron 3 objetos diferentes, se abrió el inventario, se soltó uno, se cerró y volvió a abrir.
- Resultado esperado: La UI debe mostrar exactamente los objetos actuales en el inventario con sus cantidades correctas.
- Resultado obtenido: Tras implementar un método Refresh() sincroniza perfectamente el estado del Inventory con la visualización en Inventory_UI.

4.2.3 Pruebas del Sistema de Tiles

Prueba 8: Detección de Tiles Interactuables //TODO con errores

- Objetivo: Verificar que solo se pueden interactuar tiles marcados como interactuables.
- Procedimiento: Se intentó interactuar con un tile normal y con un tile interactuable.
- Resultado esperado: Solo el tile interactuable debe cambiar de estado.
- Resultado obtenido: algunos tiles adyacentes se modifican

Prueba 9: Dirección de Interacción //TODO con errores

- Objetivo: Verificar que la interacción ocurre en la dirección hacia la que mira el jugador.
- Procedimiento: Se colocó el jugador entre 4 tiles interactuables (arriba, abajo, izquierda, derecha), se movió en cada dirección y se pulsó espacio.

- Resultado esperado: Solo debe interactuar el tile en la dirección de movimiento.
- Resultado obtenido: No funciona correctamente

4.2.4 Pruebas de Integración de Sistemas

Prueba 10: Flujo Completo de Juego

- Objetivo: Verificar que todos los sistemas funcionan correctamente en conjunto.
- Procedimiento: Se realizó una sesión de juego de 5 minutos realizando todas las acciones disponibles: mover, recoger objetos, interactuar con tiles, abrir/cerrar inventario, soltar objetos.
- Resultado esperado: No deben ocurrir errores, el rendimiento debe mantenerse estable (>30 FPS), y todas las acciones deben funcionar correctamente.
- Resultado obtenido: Al inicio se pudo detectar fallos en los botones del menú principal y los paneles de inventario y settings se superponían porque se iniciaban visibles. Posteriormente se definieron ocultos en Awake(). No se detectaron errores ni caídas de rendimiento. La integración entre sistemas es fluida.

Prueba 11: Persistencia de Estado

- Objetivo: Verificar que el estado del inventario persiste al cerrar y reabrir (preparación para sistema de guardado).
- Procedimiento: Se recogieron varios objetos, se cerró el inventario, se movió el personaje, se volvió a abrir.
- Resultado esperado: El inventario debe mantener exactamente los mismos objetos.
- Resultado obtenido: El inventario mantiene su estado correctamente durante la sesión de juego.

4.2.5 Pruebas de Integración – Sistema de Base de Datos

Prueba 12: Creación de Partida

- Objetivo: Verificar que se crea correctamente una nueva partida en la base de datos con todos sus datos asociados.
- Procedimiento: Desde el menú principal, se introduce un nombre de partida y un nombre de jugador, y se pulsa 'Confirmar'.
- Resultado esperado: Se crea un registro en la tabla SaveSlots con una semilla única (island_seed), se crea un registro en Players con posición inicial (0,0,0), y se carga la escena de juego.
- Resultado obtenido: La partida se crea correctamente y todos los datos se insertan en la base de datos. La consola muestra: '[DB] Slot creado con ID: X' y '[Save] Nueva partida creada'.

Prueba 13: Carga de Partida Existente

- Objetivo: Verificar que se cargan correctamente los datos de una partida guardada previamente.
- Procedimiento: Desde el menú de carga, se selecciona una partida existente y se pulsa 'Cargar'.
- Resultado esperado: Se cargan los datos del jugador (nombre, posición), se aplica la configuración visual de la isla usando la semilla guardada, y se carga la escena de juego.
- Resultado obtenido: Los datos se cargan correctamente y la isla muestra los mismos colores que cuando se creó la partida. La consola muestra: '[Save] Partida X cargada con isla: [nombre]'.

Prueba 14: Eliminación de Partida con CASCADE

- Objetivo: Verificar que al eliminar una partida se borran automáticamente todos sus datos relacionados (jugador, inventario, granja) gracias a ON DELETE CASCADE.
- Procedimiento: Se selecciona una partida, se pulsa 'Borrar', y se confirma la eliminación.
- Resultado esperado: Se elimina el registro de SaveSlots y automáticamente se eliminan todos los registros relacionados en Players e Inventory. La lista de partidas se actualiza.
- Resultado obtenido: La partida desaparece de la lista y la consola muestra: '[DB] Slot X eliminado con todos sus datos'. Se verificó en la base de datos que no quedan registros huérfanos.

5.2.2 Pruebas de Integración – Sistema de Islas y viajes

Prueba 15: Consistencia de Colores entre Sesiones

- **Objetivo:** Verificar que una misma partida siempre genera los mismos colores independientemente de cuándo se cargue.
- **Procedimiento:** Se crea una partida, se anota el color de la isla, se cierra el juego completamente, se vuelve a abrir y se carga la misma partida.
- **Resultado esperado:** Los colores de la isla son idénticos en ambas cargas.
- **Resultado obtenido:** El uso de `Random.InitState(slotId * 1000)` garantiza que la misma semilla genera siempre la misma secuencia de números aleatorios. La consola muestra el mismo valor RGB en ambas sesiones.

Prueba 16: Unicidad de Colores entre Partidas

- **Objetivo:** Verificar que cada partida tiene una paleta de colores visualmente diferenciable de las demás.
- **Procedimiento:** Se crean 10 partidas consecutivas y se comparan visualmente sus colores de isla.
- **Resultado esperado:** Cada isla tiene una paleta única y visualmente distintiva.
- **Resultado obtenido:** La multiplicación por 1000 de la semilla proporciona suficiente separación entre valores consecutivos. Las 10 partidas muestran colores claramente diferenciables.

5.2.3 Pruebas de Interfaz – Menú Principal

Prueba 17: Validación de Formulario Nueva Partida

- **Objetivo:** Verificar que no se puede crear una partida con campos vacíos.
- **Procedimiento:** Se pulsa 'Nueva Partida', se dejan los campos vacíos y se pulsa 'Confirmar'.
- **Resultado esperado:** Aparece un mensaje de error "Por favor, rellena todos los campos" y no se crea ninguna partida.
- **Resultado obtenido:** El mensaje de error se muestra correctamente y no se llama a `NewGame()`.

Prueba 18: Recálculo Dinámico de Layout del ScrollView

- **Objetivo:** Verificar que la lista de partidas se redimensiona correctamente cuando tiene muchos elementos.
- **Procedimiento:** Se crean 15 partidas para que superen la altura del viewport del ScrollView.
- **Resultado esperado:** El ScrollView permite desplazarse verticalmente para ver todas las partidas.
- **Resultado obtenido:** Tras muchas modificaciones con los anchors y los pivot, hubo que hacerlo via código. El uso de `Canvas.ForceUpdateCanvases()` y `LayoutRebuilder.ForceRebuildLayoutImmediate()` garantiza que el `ContentSizeFitter` calcula correctamente la altura total. El scroll funciona correctamente.

Prueba 19: Prevención de Listeners Duplicados

- **Objetivo:** Verificar que al refrescar la lista de partidas no se acumulan listeners en los botones.
- **Procedimiento:** Se abre el menú de carga, se vuelve atrás, se vuelve a abrir (para refrescar la lista), y se pulsa el botón 'Cargar' de una partida.
- **Resultado esperado:** La partida se carga una sola vez.
- **Resultado obtenido:** El uso de `RemoveAllListeners()` antes de `AddListener()` previene la acumulación. La acción se ejecuta exactamente una vez.

4.2.5 Resumen de Resultados de Pruebas

Aún hay que corregir los problemas con las tiles interactivables

Faltan aún las pruebas del menú de ajustes

No se han detectado errores críticos ni problemas de rendimiento. El proyecto cumple con todos los requisitos funcionales establecidos en la fase de diseño.

7. Referencias

Protección de datos y privacidad

- [Reglamento \(UE\) 2016/679 — RGPD](#)
- [Ley Orgánica 3/2018 — LOPDGDD](#)
- [AEPD — Guía para el cumplimiento del RGPD](#)

Propiedad intelectual

- [Real Decreto Legislativo 1/1996 — Ley de Propiedad Intelectual](#)

Licencias de software

- [Apache License, Version 2.0 — texto oficial](#)
- [Open Source Initiative — definición de licencia de código abierto](#)

Tecnologías utilizadas

- [Unity — documentación oficial](#)
- [SQLite — documentación oficial](#)
- [System.Data.SQLite — documentación del paquete](#)
- [Patrón Singleton en Unity — Game Programming Patterns](#)

Sprite pack

- [Sprout Lands Asset Pack — CupNooble](#)

Tutoriales en vídeo

- [Awesome Tuts \(2021\). Learn Unity - Beginner's Game Development Tutorial](#)
- [Digestible \(2021\). Using SQLite in Unity](#)
- [JacquelynneHei \(2022\). How to Make a 2D RPG in Unity](#)
- [Sunny Valley Studio \(2022\). Using Tools - Farm Game In Unity Devlog 2](#)

Gestión de la base de datos

- [System.Data.SQLite Documentation](#)
- [Using SQLite in Unity - Digestible](#)

Elementos de la UI

- [_ Vertical Layout Group](#)

8. Anexos

Diagrama 1: Diagrama de clases: Gameplay

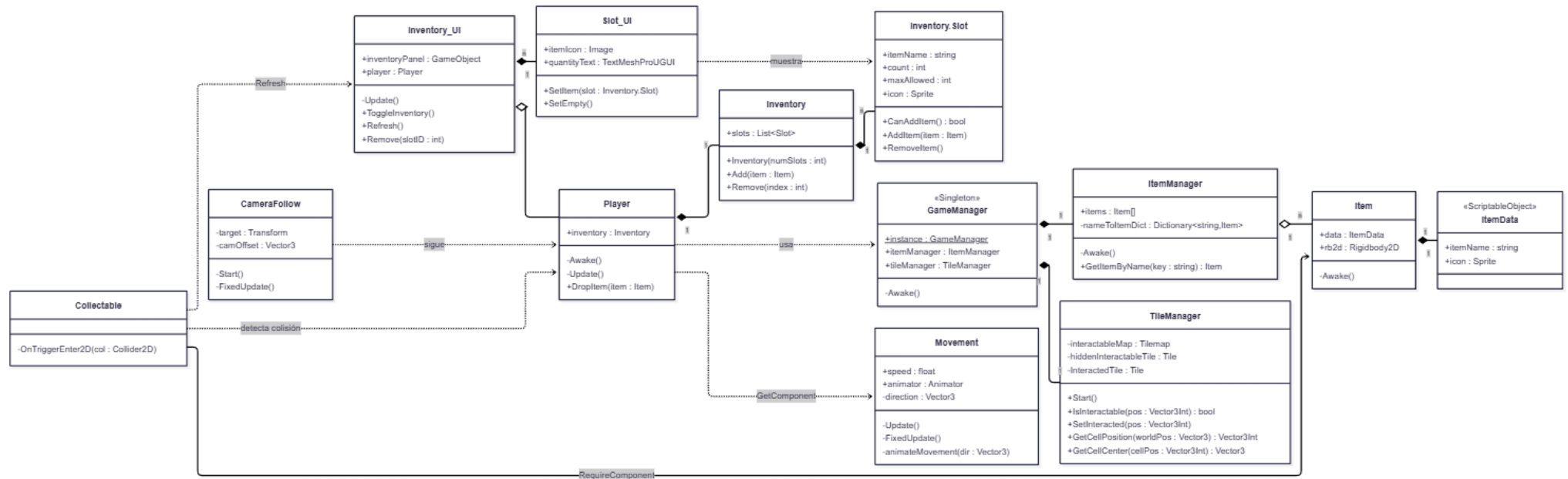


Diagrama II: Diagrama de Flujo

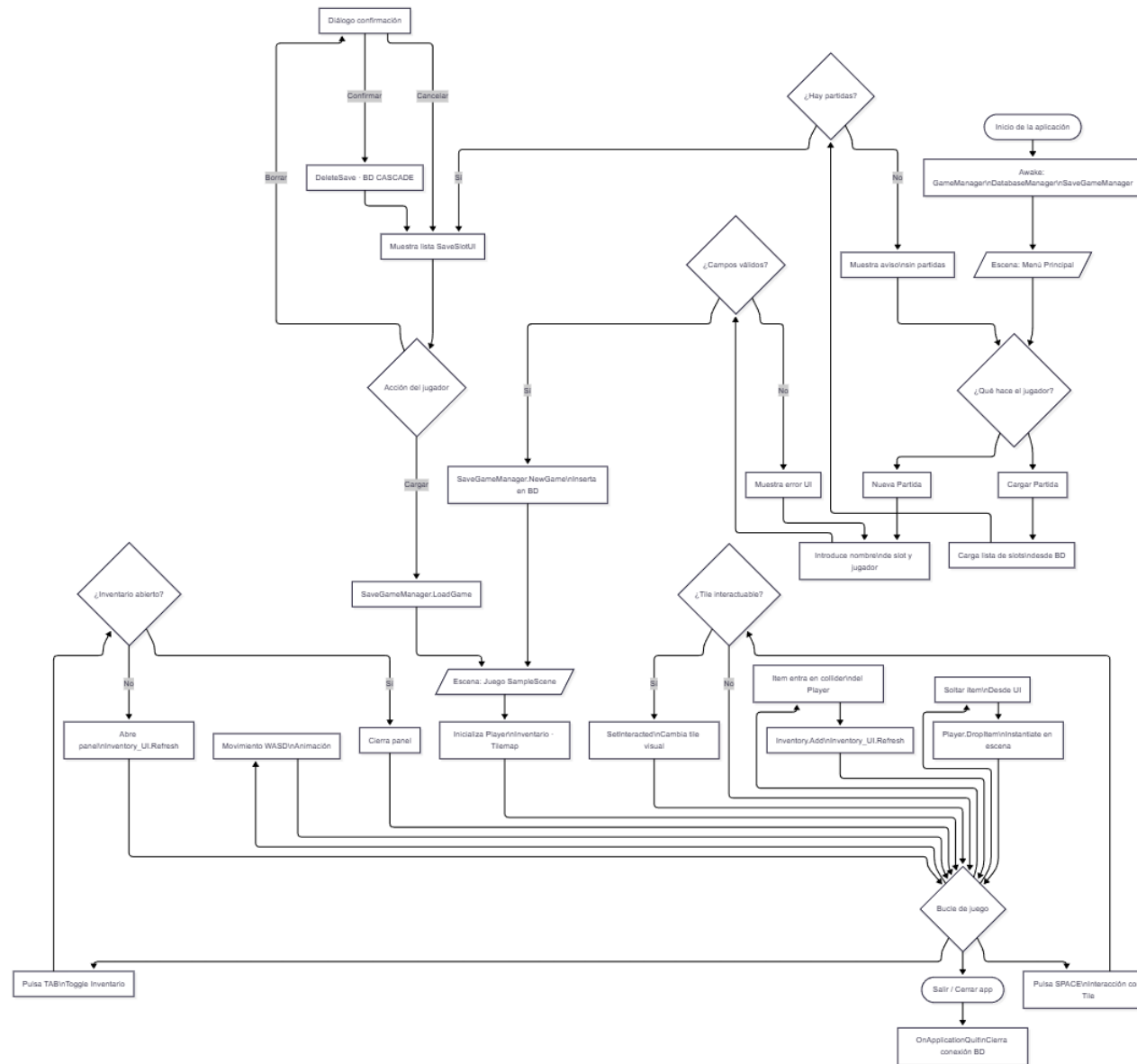
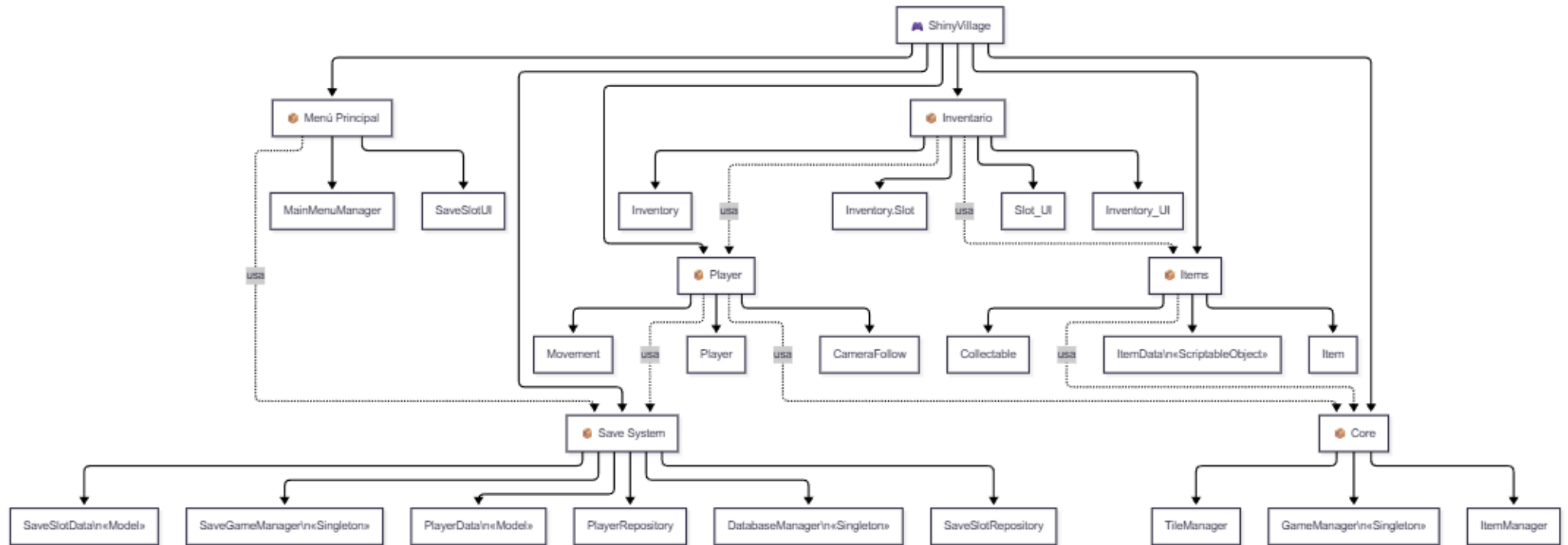


Diagrama III: Descomposición en módulo



Script de creación de la Base de datos

```
-- =====
-- SCRIPT DE CREACIÓN DE TABLAS – savegame.db
-- =====

CREATE TABLE IF NOT EXISTS SaveSlots (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    slot_name     TEXT      NOT NULL,
    player_name   TEXT      NOT NULL,
    created_at    TEXT      NOT NULL,
    last_saved    TEXT      NOT NULL,
    play_time     REAL      DEFAULT 0,
    island_seed   INTEGER DEFAULT 0
);

CREATE TABLE IF NOT EXISTS Players (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    slot_id       INTEGER NOT NULL REFERENCES SaveSlots(id) ON DELETE CASCADE,
    name          TEXT      NOT NULL,
    pos_x         REAL      DEFAULT 0,
    pos_y         REAL      DEFAULT 0,
    pos_z         REAL      DEFAULT 0
);

CREATE TABLE IF NOT EXISTS Farms (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    slot_id       INTEGER NOT NULL REFERENCES SaveSlots(id) ON DELETE CASCADE,
    farm_name     TEXT      NOT NULL,
    size_x        INTEGER DEFAULT 10,
    size_y        INTEGER DEFAULT 10,
    unlocked      INTEGER DEFAULT 1
);

CREATE TABLE IF NOT EXISTS FarmTiles (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    farm_id       INTEGER NOT NULL REFERENCES Farms(id) ON DELETE CASCADE,
    tile_x        INTEGER NOT NULL,
    tile_y        INTEGER NOT NULL,
    soil_state    TEXT      DEFAULT 'dry',
    crop_id       TEXT      DEFAULT '',
    growth_stage  INTEGER DEFAULT 0,
    days_planted  INTEGER DEFAULT 0
);

CREATE TABLE IF NOT EXISTS Inventory (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    slot_id       INTEGER NOT NULL REFERENCES SaveSlots(id) ON DELETE CASCADE,
    item_id       TEXT      NOT NULL,
    item_name     TEXT      NOT NULL,
    quantity      INTEGER DEFAULT 1,
    slot_index    INTEGER DEFAULT -1,
    item_data     TEXT      DEFAULT '{}'
);
```